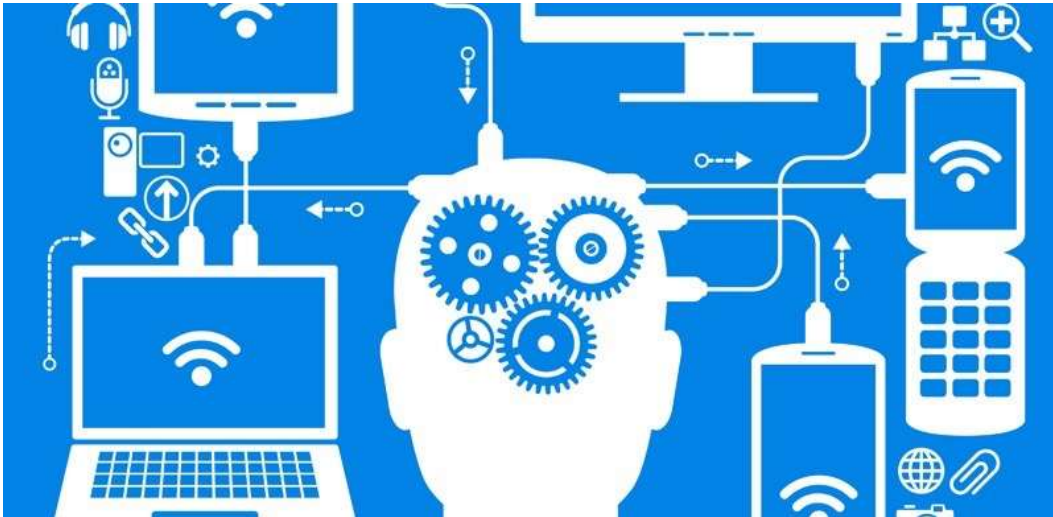




Recitations

- *Week 0: Home assignment 0 (not graded): Introduction to OOP in Java.*
- **Week 1: Home assignment 1:** Project setup and implementation of geometric primitives.
- *Week 2: Home assignment 2: Implementation of unit tests for geometric primitives (JUnit). Implementation of normal vector computation for different geometries and corresponding unit tests.*
- *Week 3: Home assignment 3: Implementation of ray-geometry intersections and corresponding unit tests.*
- *Week 4: Home assignment 4: Implementation of the Camera class, construction of rays through the view plane and unit testing of the camera.*
- *Week 5: Home assignment 5: Implementation of Scene, Ambient Light, ImageWriter, and Render classes, with corresponding unit tests.*
- *Week 6: Home assignment 6: Implementation of Emission light and adding material support.*
- *Week 7: Home assignment 7: Implementation of light propagation model for directional lights, point lights and spotlights, implementation of the Phong reflection model with multiple light sources.*
- *Week 8: Home assignment 8: Implementation of shadow rays, reflection and refraction.*
- *Week 9: Mini-project 1: Implementation of an image enhancement algorithm.*
- *Week 10-11: Mini-project 2: Implementation of a performance improvement algorithm.*
- *Week 12-13: Final mini-project presentations*

שקף זה מציג את המבנה פרויקט הקורס.

הרנדרר נבנה בהדרגה לאורך הסמסטר. כל מטלה מוסיפה יכולת חדשה למערכת ומתבססת על התשתית שנבנתה במטלות הקודמות. בתחילה נבנה את הבסיס המתמטי והגיאומטרי של המערכת – נקודות, וקטורים, קרניים, וחישוב נורמלים. לאחר מכן נוסיף חישובי חיתוכים, קרני מצלמה ומבנה הסצנה. בהמשך נוסיף מודלי תאורה וחומרים, צללים, החזרות ושבירות אור. בסיום הפרויקט נעבוד על שיפור איכות התמונה ושיפור ביצועי הרנדר.

Home Assignment 1

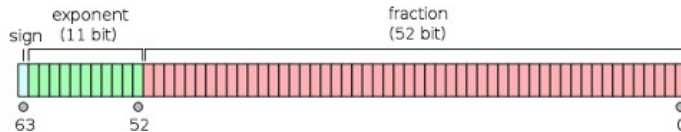
Implementation of geometric primitives and their operations,
Testing the operations using a **main** program,
Definition of geometric bodies.

בשלב הראשון של הפרויקט נבנה את התשתית המתמטית והגיאומטרית של המערכת :
נממש את הפרימיטיבים הגיאומטריים הבסיסיים ואת הפעולות הווקטוריות הדרושות לעבודה
איתם, ונבדוק את הפעולות באמצעות תוכנית ראשית.
בנוסף נגדיר את הגופים הגיאומטריים שישמשו בהמשך לחישובי חיתוך ולחישובי תאורה.
חשוב להקפיד על מימוש מדויק כבר בשלב זה, משום שכל מנוע הרינדור שייבנה בהמשך נשען
על חישובים בסיסיים אלה.

Taking Constraints in Account

Utility Class

```
public final class Util {  
    // It is binary, equivalent to ~1/1,000,000 in decimal (6 digits)  
    private static final int ACCURACY = -40;  
  
    // double store format: seee eeee eeee (1.)mmmm ... mmmm  
    // 1 bit sign, 11 bits exponent, 53 bits (52 stored) normalized mantissa  
    private static int getExp(double num) {  
        return (int)((Double.doubleToRawLongBits(num) >> 52) & 0x7FFL) - 1023;  
    }  
  
    public static boolean isZero(double number) {  
        return getExp(number) < ACCURACY;  
    }  
    public static double alignZero(double number) {  
        return isZero(number) ? 0.0 : number;  
    }  
    ... and more ...  
}
```



מחלקת Utility זו **ניתנת לכם** כחלק מתשתית הפרויקט **והיא כבר ממומשת ונבדקה**. אין צורך לממש או לשנות אותה.

מטרתה להתמודד עם מגבלות הדיוק של חישובי נקודה צפה. חישובים גיאומטריים עשויים להניב ערכים הקרובים מאוד, אך אינם מדויקים בגלל שגיאות עיגול. המתודות במחלקה מאפשרות לזהות ערכים כאלה ולהתייחס אליהם כשווים בעת הצורך. שימוש נכון בהן מסייע למנוע שגיאות חישוב בהמשך הפרויקט. חשוב להשתמש במתודות אלו במקום להשוות ישירות לאפס, שכן השוואה כזו עלולה לגרום לשגיאות שקשה לאתר בשלבים מתקדמים יותר של המערכת.

Taking Constraints in Account

```
public record Double3 (double _d1, double _d2, double _d3) {
    public static final Double3 ZERO = new Double3(0, 0, 0);
    public static final Double3 ONE = new Double3(1, 1, 1);

    public Double3(double value) { this(value, value, value); }

    @Override
    public String toString() { return "(" + _d1 + ", " + _d2 + ", " + _d3 + ")"; }

    @Override
    public boolean equals(Object obj) {
        return this == obj || (obj instanceof Double3(double od1, double od2, double od3))
            // uses Util.isZero()
            && isZero(_d1 - od1)
            && isZero(_d2 - od2)
            && isZero(_d3 - od3);
    }

    @Override
    public int hashCode() { return Objects.hash(_d1, _d2, _d3); }

    // ... Operations ...
}
```

Double3 "immutable"
ZERO:Double3 = (0,0,0) # ONE:Double3 = (1,1,1) + d1 double + d2 double + d3:double
+ equals(Object):boolean + toString():String + add(Double3):Double3 + subtract(Double3):Double3 + product(Double3):Double3 + scale(double):Double3 + reduce(double):Double3

מחלקה זו **ניתנת לכם** כחלק מתשתית הפרויקט והיא **כבר ממומשת ונבדקה**, לכן אין צורך לממש או לבדוק אותה.

האובייקטים של המחלקה הם immutable - כלומר, לאחר יצירתם לא ניתן לשנות את ערכיהם. מסיבה זו אין צורך ליצור העתקים של אובייקטים בעת עבודה איתם. הדבר ממומש באמצעות שימוש ב-**record**. השימוש ב-record מספק גם בנאי מרומוז שמקבל את הפרמטרים המתאימים לשדות המוגדרים בכותרת המחלקה. בנוסף מופיע כאן עוד בנאי, שיהיה שימושי בשלבים הבאים של הפרויקט.

שימו לב במיוחד למימוש של המתודה **equals**. בחלק הראשון של ביטוי ה-return מתבצעת גם בדיקה של טיפוס האובייקט להשוואה וגם יצירת משתנה מטיפוס Double3 באמצעות המרה אוטומטית. ניתן להשתמש במשתנה זה בהמשך אותו הביטוי.

לבסוף, במחלקה מוגדרים גם שני קבועים שימושיים: ZERO - שישמש כבר בשלב זה, ו-ONE - שישמש בשלבים מתקדמים יותר של הפרויקט.

Code Reuse by Delegation

```
public class Point {
    public static final Point ZERO = new Point(Double3.ZERO);
    protected final Double3 _xyz;
    ...
    @Override
    public String toString() { return "" + _xyz; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        return _xyz.equals(((Point) obj)._xyz);
    }

    @Override
    public int hashCode() {
        return _xyz.hashCode();
    }
}

public final class Vector extends Point { // ...
    @Override
    public String toString() { return "->" + super.toString(); }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        return super.equals(obj);
    }
    // No need to override hashCode() - it's inherited from Point
}

public final class Ray { // ...
    @Override
    public String toString() { return "Ray:" + _origin + _direction; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Ray other = (Ray) obj;
        return origin.equals(other._origin) && direction.equals(other._direction);
    }

    @Override
    public int hashCode() {
        return Objects.hash(_origin, _direction);
    }
}
```

במימוש המתודות `toString`, `hashCode`, `equals` יש להשתמש בתבנית קבועה. תבנית זו היא תבנית חובה ויש להקפיד עליה.

בתחילת `equals` מופיעות בדיקות סטנדרטיות החוזרות בכל מימוש:

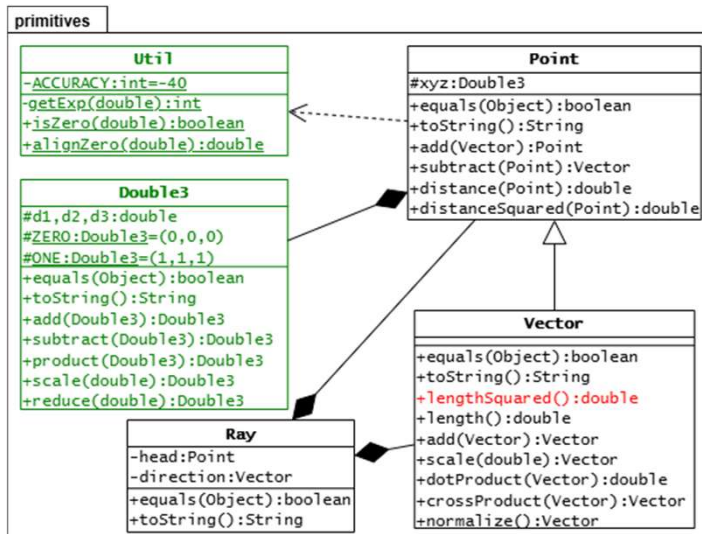
1. ראשית נבדק האם מדובר באותו אובייקט בדיוק (אותה הפניה). במקרה כזה האובייקטים שווים.

2. לאחר מכן נבדק האם האובייקט להשוואה הוא `null` או שאינו מאותו טיפוס באמצעות `getClass()`. במקרה כזה האובייקטים אינם שווים. השימוש ב-`getClass()` מבטיח שההשוואה תתבצע רק בין אובייקטים מאותה מחלקה בדיוק, ולא בין מחלקת בסיס למחלקה יורשת. כך נשמרת סימטריה נכונה של פעולת ההשוואה. לאחר בדיקות אלו ניתן להניח שהאובייקט הוא מהטיפוס המתאים ולבצע המרה לטיפוס הנכון.

ההשוואה עצמה אינה מתבצעת ישירות על ערכי השדות, אלא באמצעות האצלה (delegation) לאובייקטים של השדות עצמם – למשל קריאה ל-`equals` של `_xyz`, או שימוש ב-`super.equals()` כאשר חלק מההשוואה כבר ממומש במחלקת האב. גישה זו מונעת חזרה מיותרת על קוד ומיישמת את עקרון DRY.

חשוב גם לזכור כי `equals` ו-`hashCode` חייבות להיות עקביות: אם שני אובייקטים נחשבים שווים לפי `equals`, עליהם להחזיר גם אותו ערך `hashCode`.

Packages Design



- All classes are immutable
- Each operation returns a new object
- Zero Vector throws exception **IllegalArgumentException**
Zero vector is forbidden (no direction, normalization undefined)
- DRY principle => software reuse
- Ray's ctor normalizes its **direction** vector

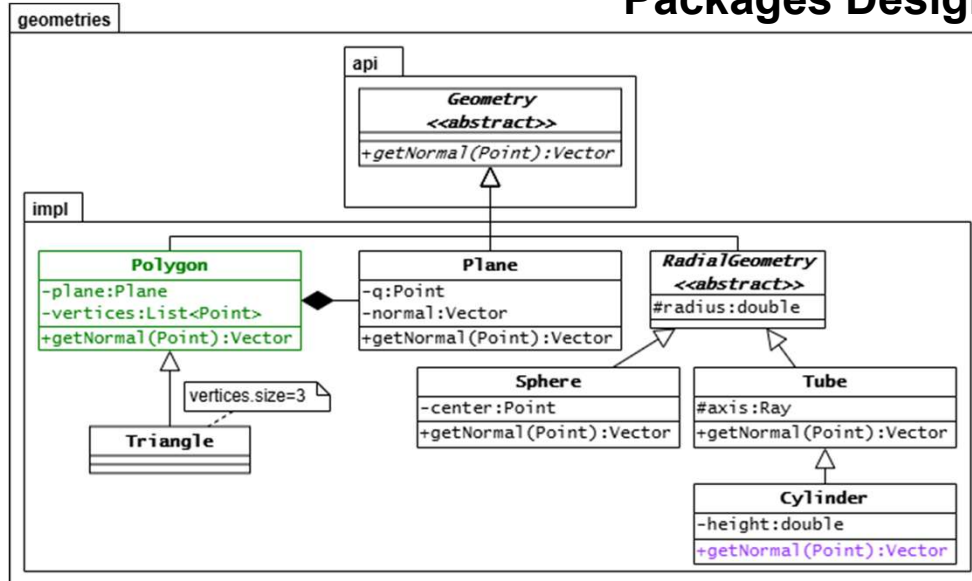
כל האובייקטים של פרימיטיבים מוגדרים כבלתי משתנים (**immutable**), כדי להבטיח שבירות נמוכה (low fragility) בהמשך הפיתוח. לכן כל השדות מוגדרים **final**, וכל פעולה על האובייקט מחזירה אובייקט חדש שהוא תוצאת הפעולה.

אין לממש מחדש את מתודת חיסור הווקטורים (**subtract**), משום שהיא כבר מממשת במחלקת האב **Point**.

לעיתים קרובות נדרש לחשב את ריבוע ערך אורך הווקטור. לכן מספקים מתודה **lengthSquared**. מתודה **length** תשתמש בה ותחשב את השורש והיא תזמן רק כאשר יש בכך צורך. כך נשמרים גם הביצועים וגם העיקרון **DRY**. יש לנקוט באותה הגישה לגבי המתודות **distance** ו-**distanceSquared** במחלקת **Point**. **בפריקט של הקורס הזה אסור ליצור וקטור אפס (0,0,0)**. לכן בבנאי של **Vector** חובה לבדוק שלא נוצר ווקטור כזה. יש לבצע את הבדיקה באמצעות השוואה של שלישיית הקואורדינטות מטיפוס **Double3** לקבוע המתאים במחלקת **Double3**. **שימו לב**: וקטור אפס אינו מייצג כיוון ולכן אינו תקין במערכת. בנוסף, פעולות רבות על וקטורים (כגון נרמול) אינן מוגדרות עבור וקטור אפס, ולכן אנו מונעים יצירתו כבר בשלב הבנאי. כל השוואות של מספרים ל-0 מתבצעות בעזרת מתודות של **Util**: **isZero** במקום **==**, ו-**alignZero** כאשר נדרשת השוואה יחסית: $\geq$, $\leq$, $>$, $<$.

במימוש מתודות יש להימנע משכפול קוד (אנטי-תבנית "העתק-הדבק") ולשמור על העיקרון **DRY**. במידת האפשר השתמשו במתודות שכבר קיימות במחלקת **Double3**! במחלקות **Point** ו-**Vector** יש לממש שני הבנאים: אחד שמקבל שלושה מספרים עבור הקואורדינטות, ואחד שמקבל אובייקט של **Double3**. במחלקת **Ray** יש לנרמל את וקטור הכיוון לפני שמירתו, משום שאין הבטחה שהווקטור המתקבל בפרמטר כבר מנורמל.

Packages Design



- All classes are **immutable**
- All **getNormal** implementations return **null**,
except Plane, which returns its normal
- Plane ctor's: 1) by 3 points, 2) by point and vector

חבילת הגופים הגיאומטריים מחולקת לשתי תת-חבילות לפי העיקרון DIP : תת-חבילה

אבסטרקטית **api** ותת-חבילה קונקרטית **impl**.

גם במחלקות של הגופים כל האובייקטים הם **immutable**, כדי להבטיח שבירות נמוכה (low fragility) בהמשך הפיתוח. לכן כל השדות מוגדרים **final**, וכל מתודה שמחזירה תוצאה מחזירה אובייקט חדש.

אין לממש מתודות השוואה (equals, hashCode) לגופים הגיאומטריים, וגם אין לממש מתודות גישה (getters) לשדות. מימוש toString אינו נדרש אך מומלץ לצורכי דיבוג.

בשלב זה כל המימושים של getNormal הם מימושים זמניים המחזירים null, **למעט במחלקה Plane**.

במחלקה Plane המתודה getNormal מחזירה את הערך של השדה **_normal**.

ב-Plane יש לממש שני בנאים: האחד שמקבל שלוש נקודות, והשני שמקבל נקודה ווקטור נורמל. יש להשאיר את הבנאי הראשון ללא מימוש מלא **בשלב זה**. בבנאי השני יש לנרמל את וקטור הנורמל לפני שמירתו, משום שאין הבטחה שהווקטור המתקבל כבר מנורמל.

Requirements

- Students must work in pairs
- Pair programming – working together:
 - Driver –typing on the keyboard and implementing the code
 - Navigator –reviewing, thinking ahead, commenting, and suggesting improvements
 - Switch roles regularly (approximately 50% / 50%)
- Using Git is mandatory
 - Project and repository name: **ISE5786_XXXX_YYYY**
 - XXXX and YYYY – last 4 digits of each student's ID
- Using GitHub (github.com) is mandatory
 - The repository **must be private**
 - Add your teacher as a collaborator
- Create tag named PR01 on the last commit.
Submit the URL of this tag on GitHub
- Both students must understand and be able to explain all parts of the code

דרישות:

- העבודה מתבצעת בזוגות בלבד
- שיטת העבודה היא תכנות זוגות (pair programming) – שני סטודנטים עובדים יחד כל הזמן (פיזית או דרך Zoom) על מחשב של אחד השותפים. בכל רגע יש שני תפקידים:
 - * סטודנט אחד כותב את הקוד בפועל (ה-Driver – "האצבעות על המקלדת").
 - * השני צופה בתהליך, מעיר, מציע שיפורים ומתקן (Navigator)
- * מדי פעם מחליפים תפקידים. בסופו של דבר כל אחד מהסטודנטים צריך לכתוב בפועל **כמחצית מהקוד**.
- * החלפת תפקידים נעשית כך: מבצעים **commit** ו-**push** למאגר, עוברים לעבוד על המחשב של השותף השני, מתחילים משם ב-**pull**, וממשיכים לעבוד.
- * בכל מקרה **אין לעבוד לבד** ולהשאיר את השותף מחוץ לתהליך.

מאגרי Git:

- יש להגדיר מאגר Git מקומי **אצל כל אחד מהשותפים** בשם שמופיע בשקף
- בנוסף יש ליצור מאגר תואם ב-GitHub ולחבר אליו את המאגרים המקומיים:
 - * שם המאגר זהה לשם המאגר המקומי.
 - * המאגר חייב להיות פרטי (**private**)
 - * יש להוסיף את המרצה כשותף (**collaborator**).
 - * על שני הסטודנטים לתרום **commit**-ים למאגר.
 - * בצעו **commit** לעיתים קרובות – אל תצברו שינויים רבים לפני ביצוע **commit**.

הגשה:

- יוצרים **tag** בשם **PR01** על ה-**commit** האחרון להגשה
- מבצעים **push** של התג למאגר ב-GitHub.
- מעתיקים את הקישור ל-**tag** ב-GitHub לתיבת ההגשה ב-Moodle.

Coding Standards

Consistent coding style improves
readability, maintainability, and teamwork

- Use Java style only:

```
if (condition) {  
    doSomething();  
} else {  
    doSomethingElse();  
}
```
- Use meaningful names, format them as PascalCase / camelCase
 - Types – PascalCase (MyClass)
 - Variables, fields, and parameters – camelCase (myVariable)
 - Instance fields start with an underscore (_myField)
 - Static (class) fields start with s_ (s_myStaticField)
 - All constants use UPPER_CASE
- Correct indentation is mandatory
- Correct spacing is mandatory (inline and between lines)

Use the IDE auto-formatter to enforce indentation and spacing

מוסכמות קוד אחידות משפרות את קריאות הקוד, מקלות על תחזוקה, ומאפשרות עבודה יעילה בצוות.

יש להקפיד על מוסכמות הקוד של Java – חובה!

- יש להשתמש ב-Java-Style מבחינת מיקום הסוגריים המסולסלות {...} – כפי שמודגם בשקף.

- יש להשתמש בשמות משמעותיים ולהקפיד על מוסכמות השמות:

* טיפוסים (מחלקות) – **PascalCase**

* שדות, פרמטרים, משתנים מקומיים, ומתודות – **camelCase**

* שמות של שדות מופע מתחילים בקו תחתון "_"

* שמות של שדות סטטיים מתחילים ב-"s_"

* קבועים – אותיות גדולות עם קו תחתון (UPPER_CASE)

- חובה להקפיד על הזחות תקינות, ריווח תקין בתוך שורה

- יש להשתמש בשורות רווח היכן שצריך לשיפור הקריאות. אך אין להוסיף רצף של שורות

רווח – מותרת שורת רווח אחת בלבד

נ.ב. פרמוט אוטומטי של הקוד פותר את רוב בעיות הסגנון:

• ב-Eclipse: Ctrl+Shift+F

• ב-IntelliJ IDEA: Ctrl+Alt+L

בהנחיות השלב מופיעות הנחיות ספציפיות להגדרות אוטו-פירמוט בשמירת קובץ והגדרות מועילות נוספות עבור שתי סביבות הפיתוח המומלצות בקורס.

Documentation

- Each class and each member (method or field) must have a **JavaDoc**-style comment
 - JavaDoc is part of the public interface documentation
 - See example in the next slide
- This documentation is **EXTERNAL**: it describes what the component is and what it does but **NOT** how it is implemented
- Write the documentation **after the declaration** but **before writing the implementation**
- **Do not** write JavaDoc for implementations of interface methods
- **Do not** write JavaDoc for `@Override` methods
- Write all comments in English only!

הקפדה על תיעוד – חובה!

- יש לכתוב תיעוד בפורמט **JavaDoc** לפני כל מחלקה ולפני כל שדה או מתודה (למעט מימושים של מתודות ממשק ומתודות נדרסות – `@Override`)
 - התיעוד הוא **תיעוד חיצוני** – כלומר הוא מתאר מה הרכיב עושה ומה מטרתו, אך **לא** כיצד הוא ממומש
 - התיעוד צריך להסביר כיצד משתמשים במתודה, מה היא מחזירה, ואילו חריגות היא עשויה לזרוק (אם יש)
 - אין לתאר בתיעוד פעולות פנימיות של המימוש (למשל לולאות, נוסחאות או אלגוריתם). התיעוד צריך לתאר את השירות שהמתודה מספקת למשתמש שלה
 - יש לכתוב את התיעוד **כאשר כותבים את הכותרת** של המחלקה או המתודה ולפני המימוש שלה. כך התיעוד נשאר נכון ועצמאי, ואינו מושפע מפרטי המימוש
 - במתודות המוצהרות בממשק (**interface**) או במחלקה **אבסטרקטית**, **JavaDoc** נכתב רק **בהצהרה** (ב-`interface` או במחלקה האבסטרקטית). אין להוסיף תיעוד למימושים של מתודות אלה במחלקות היורשות
 - כל התיעוד חייב להיכתב **באנגלית בלבד**
- בשלב זה אתם מקבלים מספר מודולים מוכנים (`Main`, `Util`, `Double3`, `Polygon`). יש לעיין בהם וללמוד מהם גם את סגנון התיעוד המצופה בקורס.
- נ.ב. ניתן להוסיף הערות בתוך קוד המימוש, אך רק כאשר הדבר באמת נדרש עקב מורכבות הקוד.

Documentation Example

```
/**
 * Immutable container for three double values.
 * <p>
 * The class is used as a basic numeric structure in the primitives package.
 * Typical uses include:
 * <ul>
 * <li>RGB color components</li>
 * <li>Point or vector coordinates</li>
 * <li>Material or attenuation coefficients</li>
 * </ul>
 * </p>
 * This class is intentionally minimal and optimized for performance,
 * as it is heavily used in geometric and color computations. It performs no
 * argument validation (such as {@code null} checks or division-by-zero checks).
 * It assumes correct usage by the calling code.
 * @param _d1 first number
 * @param _d2 second number
 * @param _d3 third number
 * @author Dan Zilberstein
 */
public record Double3(double _d1, double _d2, double _d3) {
    /** Constant triad (0,0,0) */
    public static final Double3 ZERO = new Double3(0, 0, 0);
    ...

    /**
     * Adds this triad to another triad component-wise.
     * @param rhs the triad to add
     * @return a new {@code Double3} containing the component-wise sum
     */
    public Double3 add(Double3 rhs) { return new Double3(_d1 + rhs._d1, _d2 + rhs._d2, _d3 + rhs._d3); }
    ...
}
```

JavaDoc structure

1. Description – what the class/method does
2. Tags – @param, @return, @throws, ...
3. No implementation details

בדוגמה זו מוצג תיעוד Javadoc טיפוסי למחלקה, לשדה ולמתודה.

בתחילת התיעוד מופיע תיאור קצר של המחלקה – מה היא מייצגת ומה השימושים הטיפוסיים שלה. רצוי שהמשפט הראשון יהיה קצר וברור, כי הוא מופיע בדרך כלל בתצוגות מקוצרות של התיעוד (למשל ב-IDE). לאחר מכן ניתן להוסיף פסקה נוספת עם הסבר רחב יותר, לדוגמה על מטרת המחלקה או על אופן השימוש בה.

לאחר התיאור מופיעות תגיות התיעוד, כגון @param לתיאור פרמטרים ו-@return לתיאור הערך המוחזר.

חשוב לזכור: התיעוד מתאר **מה המחלקה או המתודה עושה**, ולא **איך היא ממומשת**. מטרתו לאפשר שימוש נכון במחלקה גם בלי לקרוא את הקוד. לכן אין לתאר את האלגוריתם או פרטי המימוש. המטרה של Javadoc היא לאפשר למתכנתים אחרים להשתמש במחלקה בצורה נכונה בלי לקרוא את הקוד עצמו.

ניתן להשתמש במספר תגיות HTML פשוטות כדי לשפר את הקריאות של התיעוד, למשל <p> לפסקה חדשה או ו- לרשימה. לרוב תגית פותחת נסגרת בעזרת תגית תואמת עם /, למשל:

<p> ... </p>

אין צורך להכיר HTML מעבר לכך.